GraphModi: Updating Graph Language Models in Multi-turn Dialogue
Final Report

Prasoon Dev Yajat Lakhanpal Hrishiraj Mitra Arihant Rastogi
Team Hansraj
International Institute of Information Technology, Hyderabad

Abstract

Graph language models (GLMs) answer questions from soft graph tokens, where the graph is assumed to be fixed. In dialogue, users revise the world, and graph tokens computed before a revision become stale. We build a synthetic benchmark of multi-turn sessions (4,455 turns per split) in which each turn is a revision followed by a question, and 36% of test turns change the answer. GraphModi converts each revision into a typed edit program, applies it to an explicit graph, re-encodes the result, and answers from the new graph tokens with a frozen Llama-3.1-8B-Instruct that never sees the revision text. With a GraphToken-style reader (three seeds), re-encoding scores 54.3% on test against 41.2% for stale tokens plus the revision text, 32.5% for a shuffled graph, 34.7% for a trained soft prompt that reads the revisions, and 31.0% for zero-shot Llama reading the serialized graph. Model-written edits are execution-correct on 96.6% of turns and cost 0.1 points. The gain is largest on answer-changing turns (51.8% vs. 24.2% with stale tokens), but learned local classifiers reach 67–70% on the structural tasks, and on turns that no local rule solves every reader stays below the question alone (19–24% vs. 24.6%): the reader tracks updates through local features, not multi-hop reasoning. A frozen random GNN with a trained projector matches the jointly trained reader (55.2%), and linear probes show that its node embeddings already expose degree, status, and two-hop counts. A TEA-style reader, whose GNN is pretrained with TEA-GLM's contrastive objective and then frozen, reaches 42.1%: the pretraining removes station status from the node embeddings, although it keeps more node-attribute content than the other readers.

1 Introduction

Graph language models (GLMs) map a graph through a GNN into soft tokens that a frozen or adapted language model can attend to (Perozzi et al., 2024; Wang et al., 2024; He et al., 2024; Plenz and Frank, 2024). All such work assumes the graph does not change. Real graphs are updated, and in dialogue a user may report changes as they happen. After such a change, graph tokens computed from the old graph describe a world that no longer exists.

One way to handle this is to keep the updates in the text context and rely on the language model to track them. Dynamic-graph QA benchmarks study this setting with graphs described in text (Comrie and Kleinberg, 2025). Another is to update stored knowledge directly, but parametric knowledge editors can recall a new fact and fail to propagate its effects (Zhong et al., 2023). GraphModi keeps state external: an edit program is applied to an explicit graph, the graph is re-encoded, and the QA call receives only the new graph tokens and the question, never the revision text. This lets us score writing (did the edit land?) separately from reading (can the model answer from the new tokens?).

Research questions.

RQ1: Can a frozen LLM emit executable edit programs from a few-shot prompt, and what do its errors cost downstream?

RQ2: Does answering from re-encoded tokens beat stale G_0 tokens and text-history controls as turns accumulate, especially on answer-changing turns?

RQ3: Do graph tokens carry information beyond graph-blind baselines, and beyond what local features of the queried stations give?

Contributions.

i) A formulation of multi-turn GLM updating with an edit language and executor, which separates write accuracy from read accuracy.

ii) A multi-turn benchmark across graph size, density, session length, and task, with paired conditions that isolate stale tokens, text history, graph-blind and zero-shot text baselines, and learned local-shortcut ceilings.

iii) Evidence over three seeds that re-encoding beats stale tokens and every text channel, by the widest margin on answer-changing turns, and that model-written edits cost almost nothing.

iv) A diagnosis of what the readers learn: a frozen random GNN matches a jointly trained one, both read local features only, and TEA-GLM contrastive pretraining discards the node status that updates change while keeping node-text content.

2 Related Work

Graphs as text. A graph can be written in natural language so that an LLM attempts structural reasoning with no GNN (Wang et al., 2023; Fatemi et al., 2024). Fatemi et al. (2024) show that accuracy varies substantially with how the graph is encoded as text, and Wang et al. (2023) find that LLM graph reasoning can rely on spurious patterns instead of the actual relationships in the graph. We evaluate the serialized graph as a zero-shot baseline, and several of our controls use text history.

Soft graph tokens. GraphToken learns a graph encoder end-to-end through the language-modelling objective with a frozen LLM (Perozzi et al., 2024). TEA-GLM pretrains a GNN with instance-wise graph contrastive learning (Zhu et al., 2020) plus a feature-wise contrastive loss on the principal components of the LLM's token embeddings, freezes it, and then trains a linear projector onto a fixed number of tokens (Wang et al., 2024). G-Retriever and the graph language models of Plenz and Frank (2024) also couple graph representations with frozen or lightly adapted LMs (He et al., 2024). All of them evaluate QA against a graph supplied once per query and do not explore how a revision should modify the encoded graph. We build TEA-style and GraphToken-style readers rather than claim exact reproductions, since our readouts, tasks, and training data differ from the originals.

Do GLMs use the graph? CLEGR shows that a graph-blind soft-prompted LLM can rival full GLMs on metro reasoning, and proposes a shuffled-graph control to detect this (Petkar et al., 2026). Liu et al. (2024) ask the same question for soft prompting on graph learning tasks. We adopt the soft-prompt baseline and the shuffled-graph control, and treat parity with either as evidence against a graph-reading claim.

Dynamic graphs and stateful dialogue. DyGraphQA evaluates LLMs on graphs that change through additions and deletions described as text or structured data (Comrie and Kleinberg, 2025). MQuAKE shows that knowledge edits made inside model parameters often fail to propagate to multi-hop questions (Zhong et al., 2023). SKG-Eval stores conversational claims in an incremental knowledge graph to detect cross-turn conflicts (Shil and Samui, 2026).

3 Problem Formulation

Consider a transit assistant that initially knows two routes between a pair of stations. The user then says that one station is closed. The text decoder sees the correction, but a frozen graph representation still describes the station as open, so the model is given two versions of the world. Keeping the full conversation in the prompt may help, but it does not repair the graph that the GNN encodes, and it makes the system depend on implicit state tracking by the language model. Updating needs *writing* (mapping a revision to a graph change) and *reading* (answering from re-computed graph tokens), which we score separately on turns where the edit changes the answer and where it does not.

Graph and session. Let $G = (V, E, a, \rho)$ be an undirected attributed metro graph. Node attributes include line, accessibility, architecture, cleanliness, music, size, and $\mathrm{status} \in \{\mathrm{open}, \mathrm{closed}\}$; edge relations ρ are track or $\mathrm{transfer}$. A session is $S = (G_0, (r_t, q_t)_{t=1}^{T})$, where r_t is a natural-language revision and q_t a question.

Edit language. An edit program is a semicolon-separated list of at most three operations terminated by `END`. Legal operators are `SET NODE` (set station status), `ADD EDGE`, `DEL EDGE`, and `NOOP`. Gold programs $P_t^\star$ induce

$$G_t = \mathrm{Apply}(G_{t-1}, P_t^\star). \tag{1}$$

Gold and stale answers are $a_t^\star = \mathrm{Solve}(G_t, q_t)$ and $\tilde{a}_t = \mathrm{Solve}(G_{t-1}, q_t)$; the answer-changing set is $\mathcal{C} = \{t : a_t^\star \neq \tilde{a}_t\}$.

GraphModi. At each turn the system computes

$$\hat{P}_t = f_{\text{edit}}(r_t), \tag{2}$$

$$\hat{G}_t = \text{Apply}(\hat{G}_{t-1}, \hat{P}_t), \tag{3}$$

$$Z_t = W \cdot \rho\big(\text{GNN}(X(\hat{G}_t)), q_t\big) \in \mathbb{R}^{K \times d}, \tag{4}$$

$$\hat{a}_t = \text{LLM}\big(\text{prompt}(q_t),\ Z_t\big). \tag{5}$$

Edit prediction uses the revision text only; the graph is used for node-name resolution. The revision r_t is withheld from the QA call, so a correct answer after an edit has to come from the updated tokens. The LLM stays frozen; the GNN and projector are trained by minimizing $-\log p(a \mid q, Z(G, q))$ on static QA pairs.

Metrics. We report exact-match accuracy overall, on $\mathcal{C}$, and on its complement, plus execution-equivalent edit accuracy for the write path. Answers are parsed from the first line of the generation. Confidence intervals are 95% session-clustered bootstrap intervals, and differences between conditions are paired by turn and clustered by session. Two reference points are not models. The *per-task majority* predictor answers each task's most common gold answer, and the *stale answer* returns $\tilde{a}_t$, the answer before the turn's edit; it is an upper reference for a system that ignores edits.

4 Methodology

Write path. A few-shot prompt (output format, a NOOP example, multi-operation outputs separated by '`;`' and ended by '`END`') makes the frozen LLM emit an edit program with greedy decoding. Station labels, including multi-word names, are mapped to node IDs and operations are applied as given; invalid operations are skipped, so an edit error propagates to later turns. Edit accuracy is the fraction of turns where the predicted program yields the same graph as the gold program.

Read path. Nodes are featurized by a frozen `bert-base-uncased` encoder (Devlin et al., 2019) (768-d, mean-pooled over "`label: attr=val, ...`", including status) and passed through a 3-layer GraphSAGE (Hamilton et al., 2017) with sum aggregation, layer normalization, no dropout, and widths 768→1024→1024→4096. The output width equals the LLM hidden size, which TEA-GLM pretraining requires; all graph readers share this encoder shape. The readout is task-conditional: the source and target node embeddings when the question names them, and the mean pool otherwise. It is concatenated with a hop-radius scalar and a task one-hot (12,305-d) and linearly projected to K=10 tokens of width 4096. As in CLEGR, the text of the queried node(s), $W(f_i)$, is added to the question, with station status removed so that it can only come from the graph. The LLM is a frozen Llama-3.1-8B-Instruct in bfloat16 (Llama Team, AI @ Meta, 2024), and every prompt is `Question: {q}\nAnswer:` followed by the graph tokens.

Readers. *GraphToken-style*: the GNN starts from random weights and is trained jointly with the projector. *TEA-style*: two stages. The GNN is first pretrained self-supervised with TEA-GLM's objective, as in the authors' released code: two views of each batch (edges dropped with probability 0.3 and 0.4, node-feature dimensions masked with probability 0 and 0.1), a GRACE instance-wise loss (Zhu et al., 2020) through a two-layer projection head, plus a feature-wise contrastive loss on the node embeddings projected onto the top 1,000 principal components of Llama's token-embedding table (τ=0.4, Adam lr 2×10^{-3}, dropout 0.5, up to 70 epochs with patience 10). The GNN is then frozen and only the projector is trained. Two ablations sit between them: a *warm start* that initializes the GraphToken-style GNN from the TEA-GLM-pretrained one and trains it jointly, and a *frozen random GNN* that trains only the projector on a seeded random GNN. The *soft prompt* learns 10×4096 prompt tokens with no GNN.

Training. All readers train for one epoch on 24,000 static QA pairs (18,000 static items plus 6,000 before/after pairs around a random edit) with AdamW (weight decay 10^{-4}), effective batch 24, bf16, a cosine schedule with 200 warmup steps, and validation loss every 100 steps on 1,000 validation items; the checkpoint with the lowest validation loss is evaluated. For every reader, the projector learning rate is chosen from $\{10^{-3}, 3\times10^{-4}, 10^{-4}\}$ by validation loss on seed 42, and all four chose 10^{-4}; jointly trained GNNs use 10^{-4} (Appendix E). GraphToken-style and TEA-style use seeds 42, 43, and 44; the ablations and the soft prompt use seed 42. No model is trained on multi-turn sessions; all must generalize from static examples to graph states created at evaluation time.

Benchmark. Graphs are a ring core with pendant chains (some closing into triangles), 12–22 stations, mean degree 2.8/3.8/5.0 for the sparse,

Graph size	12–22 stations
Mean degree (sparse/med./dense)	2.8 / 3.8 / 5.0
Max diameter	7 / 6 / 5
Static eval (val / test)	2,178 / 4,356
Sessions / turns per split	1,188 / 4,455
Multi-turn tasks	9
Answer-changing turns (test)	36.4%
NOOP turns (test)	16.7%
Training pairs	24,000, 1 epoch
Seeds	3 (GT, TEA); 1 (others)

Table 1: Benchmark and training setup.

medium, and dense settings, and diameter capped at 7/6/5. Nodes carry CLEGR-like attributes and fictional two-word station names. Static QA covers eleven task families (Appendix D) with per-task answer quotas, so yes/no tasks are balanced. The multi-turn grid crosses 11 node counts, 4 session lengths $\{1, 2, 4, 8\}$, 3 densities, and 9 tasks: 1,188 sessions and 4,455 turns per split. Revisions use three paraphrase families; the `notice` family is held out of the few-shot edit prompt. A per-split quota makes 36.4% of test turns answer-changing (the stale answer scores 63.6%), 16.7% of turns are NOOP, and no session contains a net-no-op, cancelled, or disconnecting edit. Every stored label and edit was re-derived by independent solvers (0 mismatches), and a leak audit of the question text is clean.

Shortcut ceilings. Because the benchmark is synthetic, we measure how far local features of the queried stations go. A per-task gradient-boosted classifier fit on static train with one-hop features (degree, status, leaf and closed neighbours, smallest neighbour degree, hop radius, graph size) reaches 63.4% on static test and 61.9% on the updated test-session graphs over the non-attribute tasks; adding two-hop features (stations, leaves and closed stations at two hops, triangles, neighbour degrees, common neighbours; no distances) reaches 70.1% and 67.0%. A 3-layer GNN sees all of these features, so accuracy near these ceilings shows local reading, not multi-hop reasoning. The 1,052 test turns that both classifiers get wrong form the *local-wrong* slice. Text-only lookups fit on validation sessions reach 36.6% from the revision text alone and 78.7% from the stale answer plus the revision text, because 50% of turns ask about an edited station; the *changed-and-unmentioned* slice (359 answer-changing turns whose question does not name an edited station) removes that route.

Conditions. Each multi-turn session is evaluated under the paired conditions in Table 2. They differ in which graph the tokens encode and in what text the model sees. The oracle condition re-encodes the gold graph and the predicted condition re-encodes the graph built from the model's own edits, so the gap between them is the cost of edit errors.

5 Experiments

Table 1 summarizes the setup. All readers share the frozen Llama-3.1-8B decoder, the BERT node features, the encoder shape, and the same session IDs. Static QA covers eleven task families and the multi-turn grid nine. Multi-turn results are on the 4,455 test turns; the validation split is used only for checkpoint and learning-rate selection. Every number in this report is produced by the scripts in the released code from the stored per-turn outputs.

6 Results

6.1 Static evaluation

Table 3 shows single-graph QA (RQ3). GraphToken-style reaches 50.3% on test against 32.6% for the soft prompt and 29.6% for the per-task majority, and the ranking is the same on validation. Its gains are on structural tasks: node degree 99.1% (soft prompt 11.6%), cycle membership 79.0% (51.8%), reachability 65.7% (54.0%), edge existence 61.2% (45.7%), and shortest path 33.6% (14.1%). These are also the tasks a local classifier solves (static ceilings: 100, 95.7, 92.4, 95.2, and 62.1%), and GraphToken-style stays below those ceilings on every one of them. The attribute tasks stay near the soft prompt (Appendix D). The frozen random GNN is level with GraphToken-style (50.8%), while the two readers built on the TEA-GLM-pretrained GNN fall behind: the warm start reaches 46.8% and TEA-style 42.7%. The pretrained GNN changes which tasks are solved rather than lowering accuracy uniformly: TEA-style is far weaker on node degree (44.2%) and at chance on cycle membership (52.0%), but stronger on attribute lookup (46.5% vs. 29.5%) and the most common attribute within two hops (36.3% vs. 28.5%).

6.2 Multi-turn conditions

Table 4 gives the multi-turn results. For GraphToken-style, re-encoding the gold graph scores 54.3% (s.d. 0.2 over seeds; 95% CI of seed

Condition	Explanation
`oracle_updated_graph`	Graph tokens encode the gold G_t, re-encoded every turn; the prompt is the question only. The tokens describe the true current graph, with no edit-prediction error.
`predicted_updated_graph`	The frozen LLM writes an edit program from the revision text, the executor applies it to its own previous graph, and the result is re-encoded every turn; the prompt is the question only. Edit errors persist into later turns. This is the full GraphModi pipeline.
`structure_only`	Graph tokens encode the initial graph G_0 and are never updated; the prompt is the question only. Shows what the starting graph alone provides.
`frozen_graph_history`	Stale G_0 tokens with the accumulated revision text added to the question. Tests whether text can make up for stale tokens.
`shuffled_graph`	Tokens encode another session's G_0 and the prompt is the question only: right token format, wrong graph. Tests whether the tokens encode this session's graph.
`question_only`	No graph tokens and no history; the prompt is the question only.
`soft_prompt`	Trained soft prompt, no GNN; the prompt is the revision history plus the question. A graph-blind baseline that can answer many questions from the revision text.
`soft_prompt_no_history`	The same soft prompt with the question only, like-for-like with the graph conditions.
`serialized_current_graph`	Zero-shot Llama, no graph tokens; the prompt is the gold G_t written as text (stations with attributes, edges with relations) plus the question.
`serialized_initial_history`	Zero-shot Llama; G_0 as text, the revision history, and the question. The pure in-context updating baseline.
`token_matched_history`	Zero-shot Llama; the last 64 words of revision history plus the question.
`tool_solver`	A symbolic solver on the gold G_t; checks the labels (100% by construction).
stale answer	No model is run. Returns $\tilde{a}_t$, the answer before the turn's edit.
per-task majority	No model is run. Returns each task's most common gold answer.

Table 2: Multi-turn evaluation conditions. "History" is the accumulated revision text of earlier turns. The last three rows are reference points, not model conditions.

	GT	TEA	Warm	Rand.	Soft	Maj.
Validation	51.1	43.2	47.1	51.5	33.7	–
Test	50.3	42.7	46.8	50.8	32.6	29.6

Table 3: Static exact-match accuracy (%). GT: GraphToken-style; TEA: TEA-style; Warm: GraphToken-style from the TEA-GLM-pretrained GNN; Rand.: frozen random GNN; Maj.: per-task majority. GT and TEA are means over three seeds (test s.d. 0.1 for both).

Condition	GT	TEA	Warm	Rand.
oracle_updated_graph	54.3	42.1	44.5	55.2
predicted_updated_graph	54.2	42.1	44.6	55.1
frozen_graph_history	41.2	38.2	38.8	41.7
structure_only	41.2	37.9	38.0	41.7
shuffled_graph	32.5	31.9	30.8	31.8
question_only		26.5		
soft_prompt / no history		34.7 / 33.3		
serialized_current_graph		31.0		
serialized_initial_history		29.4		
token_matched_history		27.9		
per-task majority		31.8		
stale answer		63.6		

Table 4: Multi-turn test accuracy (%), 4,455 turns. GT and TEA are means over three seeds; the seed s.d. of `oracle_updated_graph` is 0.2 (GT) and 0.4 (TEA). Rows below the rule do not depend on the graph reader; the serialized and token-matched rows are zero-shot Llama.

42: 51.8–56.3), and re-encoding the graph built from the model's own edits scores 54.2%. Paired by turn, re-encoding beats the shuffled graph by 22.0 points (seed 42; CI 19.5–24.5), stale tokens plus history by 13.6 (11.7–15.5), and stale tokens alone by 13.4 (11.6–15.4), with the same picture on the other two seeds. It beats the soft prompt with the revision history by 19.4 points (16.8–22.0) and without it by 20.8 (18.2–23.3). Zero-shot Llama does no better than the per-task majority even when it reads the whole current graph as text (31.0% vs. 31.8%; prompts of up to 1,937 tokens, never truncated), and adding the initial graph and the revision history in context gives 29.4%. The stale answer scores 63.6% because 63.6% of turns keep their pre-edit answer; it is a reference for a system that ignores edits, not a competitor.

The other readers follow the same pattern at different levels (Figure 3, Appendix B). The frozen random GNN scores 55.2%, statistically level with GraphToken-style (paired with seed 42: -1.1 points, CI -2.3 to 0.0), so training the GNN jointly adds nothing measurable over a random one here. TEA-style scores 42.1%; re-encoding still beats

Condition	1	2	4	8
oracle_updated_graph	54.1	53.6	55.1	54.1
predicted_updated_graph	53.5	53.5	55.4	53.8
frozen_graph_history	47.0	40.7	41.7	40.3
structure_only	45.0	39.6	42.2	40.7
shuffled_graph	28.7	31.7	33.1	32.8
soft_prompt	30.6	31.8	35.4	35.6
serialized_initial_history	29.3	33.3	29.0	28.7
stale answer	64.0	61.1	63.3	64.3

Table 5: Test accuracy (%) by session length, GraphToken-style (mean of three seeds) and the shared baselines.

its stale tokens plus history by 3.7–4.5 points and its shuffled graph by 9.4–10.9 points on every seed, and it beats the soft prompt without history by 8.5–9.3 points, but it trails the frozen random GNN by 13.2 points (seed 42; CI 11.0–15.4), which uses the same frozen-GNN recipe. The warm start, which trains the TEA-GLM-pretrained GNN jointly, reaches 44.5%, 9.6 points below GraphToken-style from random weights (CI 7.7–11.5).

6.3 Multi-turn degradation

Figure 1 and Table 5 address RQ2. Re-encoded tokens hold their accuracy at every session length (53.6–55.1%). Stale tokens drop as soon as a session has more than one turn and do not recover with the revision text: stale tokens plus history fall from 47.0% in one-turn sessions to 40.3–41.7% in longer ones, and stale tokens alone from 45.0% to 39.6–42.2%. They do not keep declining with length because the per-split quota fixes the answer-changing share at 36–39% for every length, so each additional edit is as likely to restore an answer as to change it. The soft prompt rises slightly with session length (30.6% to 35.6%) because the accumulated revision text answers more questions directly, and zero-shot Llama with the initial graph and history in context stays between 29% and 33% at every length.

Table 6 and Figure 2 (Appendix A) split turns by whether the edit changed the answer. On unchanged turns, stale and re-encoded GraphToken-style tokens are within 5 points of each other (50.9% vs. 55.7%). On changed turns, stale tokens fall to 24.2%, below the per-task majority, while re-encoded tokens keep 51.8%: a 27.6-point gap where the edit matters, against 4.8 points where it does not. On the 359 changed turns whose question does not name an edited station, where the revision

Condition	Changed	Unchanged	Unmentioned
oracle_updated_graph	51.8	55.7	38.3
predicted_updated_graph	51.2	55.9	38.2
frozen_graph_history	24.2	50.9	26.8
structure_only	23.0	51.7	27.2
shuffled_graph	29.4	34.2	27.8
soft_prompt	32.2	36.1	24.8
serialized_initial_history	27.2	30.7	28.1
per-task majority	27.3	34.4	25.3
TEA-style oracle	37.8	44.6	26.3
TEA-style stale + history	29.1	43.4	27.5
Random GNN oracle	52.8	56.6	40.9
Random GNN stale + history	24.8	51.4	24.8

Table 6: Test accuracy (%) on answer-changing turns (1,622), unchanged turns (2,833), and answer-changing turns whose question names no edited station (359). Graph rows above the rule are GraphToken-style, means of three seeds; TEA-style rows are means of three seeds, random-GNN rows one seed.

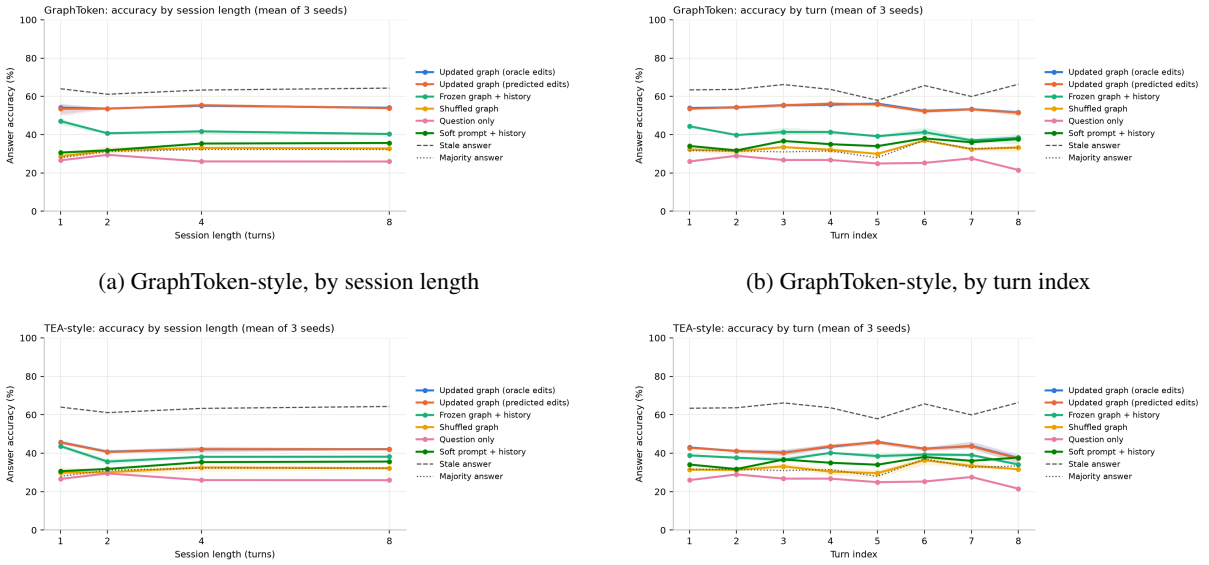
	Sparse	Medium	Dense
oracle_updated_graph	53.0	54.5	55.4
predicted_updated_graph	52.8	54.4	55.4
frozen_graph_history	39.6	41.7	42.3
shuffled_graph	32.3	33.6	31.4
soft_prompt	34.3	33.9	36.0

Table 7: Test accuracy (%) by graph density, GraphToken-style (mean of three seeds) and the soft prompt.

text cannot be matched to the question, re-encoding still scores 38.3% against 26.8% for stale tokens plus history and 24.8% for the soft prompt; here the text-only lookup that otherwise reaches 78.7% scores 2.5%. The frozen random GNN tracks edits as well as GraphToken-style (52.8% vs. 24.8% on changed turns, 40.9% on unmentioned ones). TEA-style gains 8.7 points from re-encoding on changed turns (37.8% vs. 29.1%), but on the unmentioned ones its re-encoded tokens (26.3%) are no better than its stale tokens plus history (27.5%), so its tokens barely carry the status and edge changes.

6.4 Density, task, and local shortcuts

Tables 7 and 8 break the test results down. Accuracy varies by 2.4 points across densities, and re-encoding is 21–24 points ahead of the shuffled graph at every density. By task, re-encoding helps most where the answer is a local structural fact that edits change: node degree (99.9% vs. 40.8% stale), cycle membership (83.4% vs. 62.5%), and edge existence (66.3% vs. 53.2%). GraphToken-style stays below the local ceiling on the path, cycle, and edge tasks, far below it on shortest path (25.5% vs. 55.6%) and reachability (68.4% vs. 91.3%), reaches it on the two counting tasks (34.3%



(a) GraphToken-style, by session length

(b) GraphToken-style, by turn index

(c) TEA-style, by session length

(d) TEA-style, by turn index

Figure 1: Multi-turn test accuracy by session length (left) and by turn index (right), for GraphToken-style (top) and TEA-style (bottom), means over three seeds with the seed range shaded. Dashed and dotted lines are the stale answer and the per-task majority. Later turns occur only in 8-turn sessions (297 sessions).

Task	GT	TEA	Rand.	Stale+h.	Shuf.	Soft	Maj.	Local
node_degree	99.9	49.6	97.6	40.8	11.1	11.7	13.9	100.0
cycle_membership	83.4	49.2	87.3	62.5	54.5	58.0	57.8	88.1
reachability	68.4	60.2	70.3	61.0	53.7	59.2	59.6	91.3
edge_exists	66.3	58.1	70.5	53.2	46.9	55.4	50.3	91.5
constrained_reachability	61.0	56.6	61.8	52.9	47.3	50.7	50.1	64.4
filtered_neighbor_count	34.3	30.6	33.1	30.0	20.1	18.2	19.0	33.7
most_common_attr. within hops	28.6	36.0	33.1	29.6	28.6	27.7	6.3	–
shortest_path	25.5	20.5	24.8	24.0	15.1	15.2	15.4	55.6
within_hops_count	21.3	18.6	18.4	17.0	14.7	16.6	13.9	16.8

Table 8: Multi-turn test accuracy (%) by task (n=495 each): GraphToken-style re-encoded (GT; mean of three seeds), TEA-style and frozen random GNN re-encoded (TEA, Rand.), GraphToken-style stale tokens plus history (Stale+h.), shuffled graph, soft prompt, per-task majority, and the learned local ceiling on the updated graphs (better of the one- and two-hop classifiers).

vs. 33.7%; 21.3% vs. 16.8%), and on the most-common-attribute task it equals the shuffled graph (28.6%). The frozen random GNN matches or exceeds GraphToken-style on every structural task, while TEA-style is at chance on cycle membership (49.2%), reaches only 49.6% on node degree, and is the strongest reader on the most-common-attribute task (36.0%).

The local-wrong slice makes the same point directly. On the 1,052 test turns that both local classifiers get wrong, GraphToken-style scores 22.2% re-encoded (s.d. 0.3), 21.2% with a shuffled graph, and 24.6% with the question alone; TEA-style (23.6%), the warm start (21.6%), and the frozen random GNN (19.2%) are also below the question alone. The readers therefore use the graph to track local facts such as degree, status, and nearby closures, which is what most updates change, but we find no evidence of reasoning that goes beyond a two-hop neighbourhood.

6.5 Write path

The `predicted_updated_graph` condition applies the model's own edit programs and re-encodes the result, and it answers RQ1. The frozen LLM writes an execution-equivalent program on 96.6% of test turns, and the graph state built from its own edits is exactly right on 91.1% of turns, since an error persists for the rest of a session. NOOP turns are always correct and status and deletion edits are above 97%; the weakest operation is a lone `ADD EDGE` at 79.9%. On the `notice` paraphrase family,

Node embeddings	Degree	Status	2-hop	Cos.	Rank
BERT input (no GNN)	32.2	100.0	57.9	0.97	13.0
Random frozen GNN	100.0	96.6	92.3	0.92	1.9
GraphToken-style GNN	100.0	95.8	96.9	0.97	1.9
TEA-GLM-pretrained GNN	96.4	88.1	79.2	0.24	16.2
Warm-start GNN	98.3	88.1	82.1	0.37	12.9
Majority class	36.5	88.2	59.0		

Table 9: Linear probes on frozen node embeddings (test-split graphs, accuracy %): node degree, station status, and the number of stations within two hops. Cos.: mean cosine similarity between nodes of a graph; Rank: effective rank of the embedding covariance.

held out of the few-shot prompt, edit accuracy is 91.5% against 98.9–99.3% for the other two; each task uses a single family, so this gap mixes wording with the edits that the `notice` tasks draw. These errors cost 0.1 points for GraphToken-style (54.2% vs. 54.3%; paired CI of seed 42 -0.2 to 0.4) and at most 0.1 points for every reader, and the predicted and oracle rows agree within 1.2 points by session length, change status, density, and task (Tables 5–8).

6.6 What the readers learn

To see what each GNN passes to the projector, we fit linear probes on its frozen node embeddings (multinomial logistic regression on standardized features, trained on train-split graphs and scored on test-split graphs) for three local facts that the tasks and edits depend on (Table 9). The BERT input features encode status (it is in the node text) but not degree. A frozen random GNN with sum aggregation already makes degree (100%), status (96.6%), and the two-hop count (92.3%) linearly decodable, and the GNN trained jointly by GraphToken-style adds almost nothing (100, 95.8, and 96.9%). Since the projector is a single linear map, this explains why the frozen random GNN reads as well as GraphToken-style: the information the tasks need is already linearly available before any training.

The TEA-GLM-pretrained GNN loses station status: its probe accuracy (88.1%) equals the majority class, and the two-hop count drops to 79.2%. Its instance-wise objective makes embeddings invariant to dropped edges and masked features, which removes exactly the local details that updates change; this matches TEA-style's chance accuracy on cycle membership and its weak tracking of changed answers. The loss is not embedding collapse: the TEA-GLM embeddings are the least collapsed (mean cosine 0.24, effective rank 16.2), while the random and GraphToken-style GNNs are nearly collapsed (rank 1.9) and still decode every probe target. The warm start keeps the pretrained GNN's status loss (88.1%) after one epoch of joint training while recovering degree (98.3%), which matches its multi-turn profile (node degree 72.7%, cycle membership 43.4%). In the other direction, the feature-wise objective aligns node embeddings with the principal components of Llama's token embeddings, and both readers built on this GNN are the strongest on the attribute tasks (static attribute lookup 46.5% for TEA-style and 46.2% for the warm start, against 29.5% for GraphToken-style); we read this as the pretraining keeping node-text content in a form the projector can map to word-like tokens, although we did not probe attributes directly.

7 Inferences

Graph tokens encode the specific, current graph. Re-encoded GraphToken-style tokens beat a shuffled graph by 22.0 points, a graph-blind soft prompt by 19–21 points with or without the revision text, and zero-shot Llama reading the serialized graph by 23 points, at every density and session length and on all three seeds. A model that ignored the tokens would score the same under all of these conditions, so the tokens carry session-specific information (RQ3).

Staleness hurts exactly where an edit matters. On answer-changing turns stale tokens fall to 24.2%, below the per-task majority, while re-encoded tokens keep 51.8%; on unchanged turns the two are within 5 points. The gap survives on changed turns whose question does not name the edited station (38.3% vs. 26.8%), where no text lookup helps (RQ2).

The gain comes from re-encoding, not from text history. Adding the revision text to stale tokens does not help: `frozen_graph_history` and `structure_only` are both 41.2%. Text channels that see the full history do not catch up either: the soft prompt reaches 34.7%, and zero-shot Llama with the initial graph and every revision in context reaches 29.4%. Updating the graph and re-reading it is what recovers the answer.

The write path is not the bottleneck. Model-written edits are 96.6% execution-correct and cost 0.1 points (RQ1), but revisions are templated, so this likely overstates the write path on free-form dialogue.

Status	Item
Done	Edit language, executor, few-shot edit prompt
Done	BERT/GraphSAGE read path; TEA-style (TEA-GLM pretraining), GraphToken-style and soft-prompt training
Done	Multi-turn grid, paired conditions, predicted-edit evaluation
Done	Serialized graph input (zero-shot Llama)
Done	Three seeds for GraphToken-style and TEA-style, session-clustered CIs
Done	Ablations: warm start, frozen random GNN, soft prompt without history; local-shortcut ceilings; GNN probes
Open	Contradiction probe (logical contradiction across turns)
Open	Free-form (non-templated) revisions

Table 10: Status relative to the proposal.

The reader is local. GraphToken-style accuracy stays at or below learned local ceilings on the structural tasks, and on turns that no local rule solves it is no better than a shuffled graph (22.2% vs. 21.2%) and below the question alone (24.6%). The benchmark rewards tracking local facts that updates change; it does not yet show that soft graph tokens support multi-hop reasoning.

Training the GNN adds little; pretraining it contrastively hurts. A frozen random GNN with a trained projector matches the jointly trained GraphToken-style reader (55.2% vs. 54.3%), because a random sum-aggregation GNN already exposes degree, status, and neighbourhood counts linearly. The bottleneck is the frozen LLM reading ten soft tokens, not the encoder. TEA-GLM pretraining, built for zero-shot transfer of semantic categories, makes the GNN invariant to the edge and status details that updates change; the TEA-style reader trails the frozen random GNN by 13.2 points and the warm start trails GraphToken-style by 9.6, although both keep more node-attribute content. For GLMs over graphs that change, the encoder must preserve exactly the local details that augmentation-based contrastive objectives are designed to ignore.

8 Status

Table 10 compares the finished work with the proposal. Two items remain open: a contradiction probe, in which a session evolves into a logically inconsistent state (A is the father of B, B of C, and C of A) that the GLM should flag, and revisions written freely rather than from templates.

Limitations

The benchmark is synthetic and its revisions follow templates, so edit accuracy likely overstates performance on free-form dialogue. GraphToken-style and TEA-style use three seeds; the warm start, the random GNN, and the soft prompt use one, so the tie between the random GNN and GraphToken-style rests on a single random-GNN seed. Every reader trains for one epoch on static QA, and longer training could change how the readers rank. The projector learning rate was chosen on validation loss after a first round of runs at 10^{-3}, in which the readers built on a frozen or pretrained GNN had not converged; the soft prompt's learning rate (10^{-3}) was not swept. Each multi-turn task uses a single revision paraphrase family, so wording effects cannot be separated from task effects. Our explanation of TEA-style's advantage on the attribute tasks is not backed by a dedicated attribute probe. The zero-shot text baselines use the plain question–answer template rather than Llama's chat template and see no examples of the answer formats, so they are a floor for in-context reading rather than its best case. Learned local classifiers reach 67–70% on the structural tasks, so absolute accuracy on this benchmark mostly measures local reading.

References

Cazamere Comrie and Jon Kleinberg. 2025. Can large language models reason on dynamic graphs? In *KDD 2025 Workshop on Structured Knowledge for Large Language Models*.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, Minneapolis, Minnesota. Association for Computational Linguistics.

Bahare Fatemi, Jonathan Halcrow, and Bryan Perozzi. 2024. Talk like a graph: Encoding graphs for large language models. In *International Conference on Learning Representations*.

Will Hamilton, Zhitao Ying, and Jure Leskovec. 2017. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems*.

Xiaoxin He, Yijun Tian, Yifei Sun, Nitesh Chawla, Thomas Laurent, Yann LeCun, Xavier Bresson, and Bryan Hooi. 2024. G-retriever: Retrieval-augmented

generation for textual graph understanding and question answering. In *Advances in Neural Information Processing Systems*.

Zheyuan Liu, Xiaoxin He, Yijun Tian, and Nitesh V. Chawla. 2024. Can we soft prompt LLMs for graph learning tasks? In *Companion Proceedings of the ACM Web Conference 2024*, pages 481–484.

Llama Team, AI @ Meta. 2024. The Llama 3 herd of models. *Preprint*, arXiv:2407.21783.

Bryan Perozzi, Bahare Fatemi, Dustin Zelle, Anton Tsitsulin, Mehran Kazemi, Rami Al-Rfou, and Jonathan Halcrow. 2024. Let your graph do the talking: Encoding structured data for LLMs. *Preprint*, arXiv:2402.05862.

Soham Petkar, K Hari Aakash, Anirudh Vempati, Akshit Sinha, Ponnurangam Kumaraguru, and Chirag Agarwal. 2026. A graph talks, but who's listening? rethinking evaluations for graph-language models. In *Findings of the Association for Computational Linguistics: ACL 2026*.

Moritz Plenz and Anette Frank. 2024. Graph language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 4477–4494, Bangkok, Thailand. Association for Computational Linguistics.

Avijit Shil and Suman Samui. 2026. SKG-eval: Stateful evaluation of multi-turn dialogue via incremental semantic knowledge graphs. *Preprint*, arXiv:2605.16650.

Duo Wang, Yuan Zuo, Fengzhi Li, and Junjie Wu. 2024. LLMs as zero-shot graph learners: Alignment of GNN representations with LLM token embeddings. In *Advances in Neural Information Processing Systems*.

Heng Wang, Shangbin Feng, Tianxing He, Zhaoxuan Tan, Xiaochuang Han, and Yulia Tsvetkov. 2023. Can language models solve graph problems in natural language? In *Advances in Neural Information Processing Systems*.

Zexuan Zhong, Zhengxuan Wu, Christopher Manning, Christopher Potts, and Danqi Chen. 2023. MQuAKE: Assessing knowledge editing in language models via multi-hop questions. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 15686–15702, Singapore. Association for Computational Linguistics.

Yanqiao Zhu, Yichen Xu, Feng Yu, Qiang Liu, Shu Wu, and Liang Wang. 2020. Deep graph contrastive representation learning. In *ICML Workshop on Graph Representation Learning and Beyond*.

A Accuracy on Answer-Changing Turns

B All Readers Together

C All Conditions

Figures 4–6 repeat the main-text plots with every condition, for GraphToken-style (top of each figure) and TEA-style (bottom). The stale-answer reference is 0% on answer-changing turns and 100% on unchanged turns by definition.

D Task Families

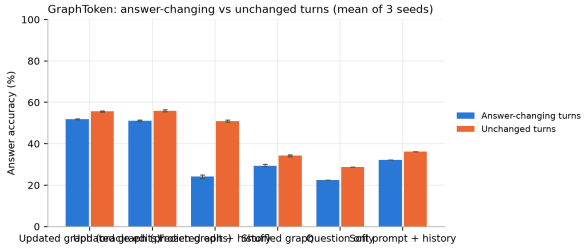
Static QA covers eleven task families; the multi-turn grid uses the first nine. Every answer is computed by a symbolic solver on the current graph, and yes/no tasks are balanced. In the definitions below, A and B are station names.

- **Edge existence**: is there a direct edge between A and B?
- **Reachability**: can B be reached from A using open stations only?
- **Constrained reachability**: can B be reached from A without passing through a station of a given class (no disabled access, no rail, or dirty)?
- **Cycle membership**: is A part of any cycle? Asked only about degree-2 stations, so degree alone does not answer it.
- **Node degree**: how many stations are directly connected to A?
- **Filtered neighbour count**: how many direct neighbours of A have a given attribute value?
- **Shortest path**: how many stations, including both endpoints, lie on the shortest open path from A to B (or `unreachable`)?
- **Within-hops count**: how many other stations within two hops of A have a given attribute value?
- **Most common attribute within hops**: what is the most common value of an attribute among the stations within two hops of A, including A itself? Only queries with a unique mode are kept.
- **Attribute lookup**: what is the value of one of A's attributes? The answer is an open categorical value.
- **Attribute check**: does A have a given attribute value?

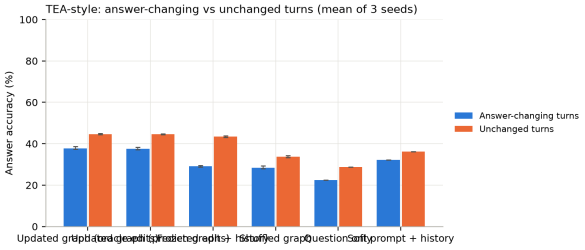
The last two are CLEGR's Facts tasks. For them, and for the most-common-attribute task, the queried attribute is withheld from $W(f_i)$, so the answer has to come from the graph tokens. The Facts tasks are static only: revisions set station status or add and remove edges, so they never change a node attribute. Reachability and shortest-path questions use open endpoints only, and the cycle task is asked only at degree-2 stations; under these filters a degree-and-status lookup rule reaches 41.0% on seven tasks, and the rule "a neighbour of a leaf is not on a cycle" reaches 79% on cycle membership.

Static accuracy by task family. Figures 7 and 8 break Table 3 down by task family against the soft prompt and the learned local ceiling. GraphToken-style is strongest on node degree (99.1%), cycle membership (79.0%), reachability (65.7%), and edge existence (61.2%), and below the local ceiling on each. On the attribute tasks it stays close to the soft prompt: attribute lookup 29.5% (soft prompt 25.8%), most common attribute 28.5% (26.5%), and attribute check 49.4% (50.5% majority), so the tokens carry little beyond the node status and structure that the training signal emphasizes. TEA-style has the opposite profile: it is weaker on every structural task (node degree 44.2%, cycle membership 52.0%, shortest path 23.0%) but the strongest reader on attribute lookup (46.5%) and the most common attribute (36.3%); attribute check stays near the majority for every reader (48.7–53.8%). The frozen random GNN follows GraphToken-style closely on every task (node degree 94.9%, cycle membership 81.8%, attribute lookup 28.0%).

E Hyperparameters



(a) GraphToken-style



(b) TEA-style

Figure 2: Test accuracy on turns where the edit changed the answer (36%) and where it did not (64%), means of three seeds; error bars are the seed range.

Item	**Value**
LLM	Llama-3.1-8B-Instruct, bf16, frozen
Node features	`bert-base-uncased`, 768-d, frozen
GNN	GraphSAGE 768/1024/1024/4096, sum, LayerNorm, dropout 0
Readout	pooled / source / target, hop scalar, task one-hot (12,305-d)
Projector	$12{,}305 \to 10 \times 4096$, linear
TEA-GLM pretraining	Adam $2{\times}10^{-3}$, $\tau{=}0.4$, dropout 0.5, edge drop 0.3/0.4, feature mask 0/0.1, 1,000 PCs, 32 graphs/batch, $\leq$70 epochs
Training	AdamW, wd 10^{-4}, eff. batch 24, 1 epoch, cosine, 200 warmup
Projector lr	10^{-4} (GT), 10^{-4} (TEA), 10^{-4} (warm), 10^{-4} (random)
GNN lr (joint)	10^{-4} (GT, warm start)
Checkpoint	lowest validation loss (every 100 steps, 1,000 items)

Table 11: Key hyperparameters. Projector learning rates were chosen per reader from $\{10^{-3}, 3{\times}10^{-4}, 10^{-4}\}$ by the best validation loss on seed 42.

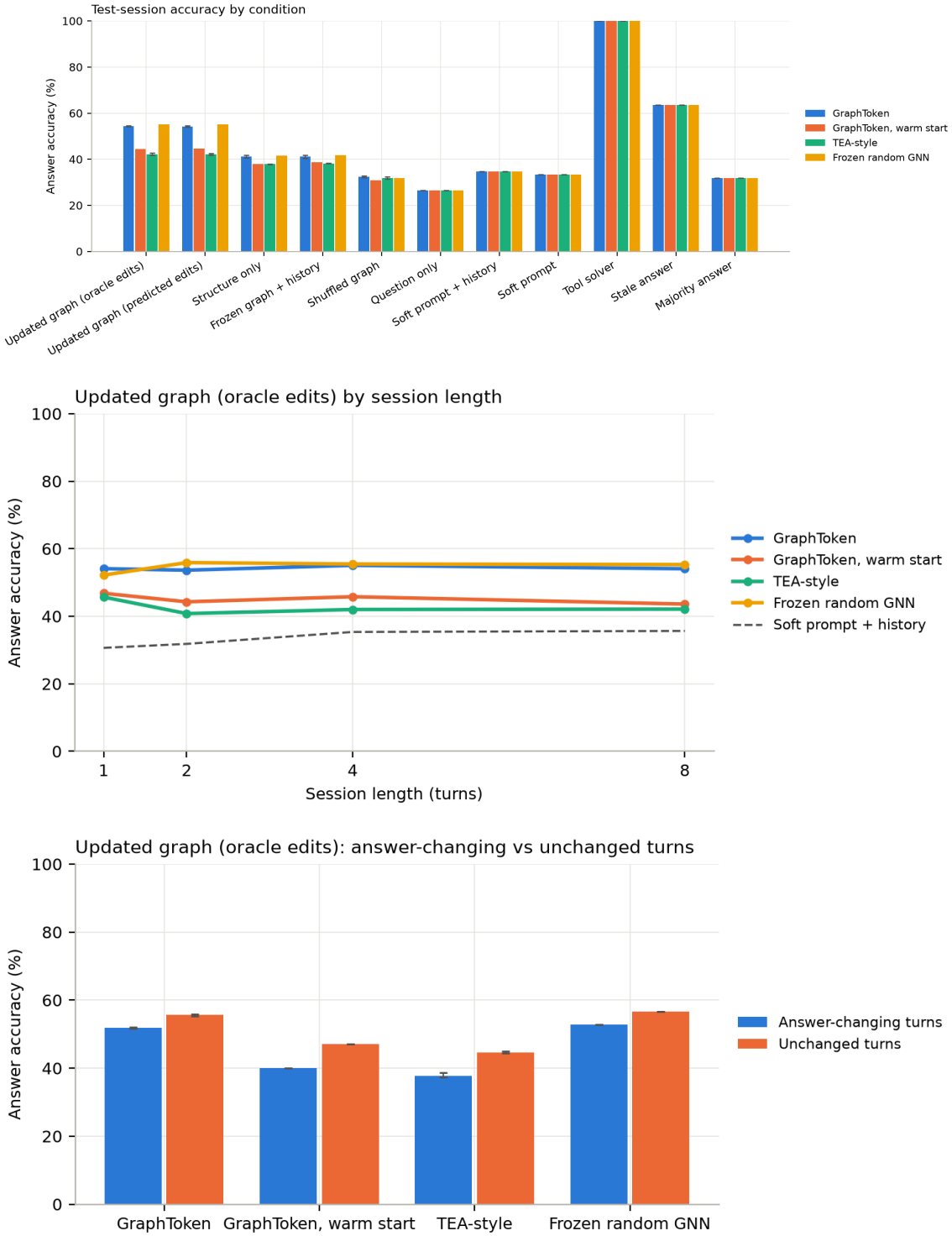


Figure 3: Test results for all four graph readers. Top: accuracy under each condition. Middle: `oracle_updated_graph` by session length, with the soft prompt. Bottom: `oracle_updated_graph` on answer-changing and unchanged turns. GraphToken-style and TEA-style are means of three seeds (error bars: seed range).

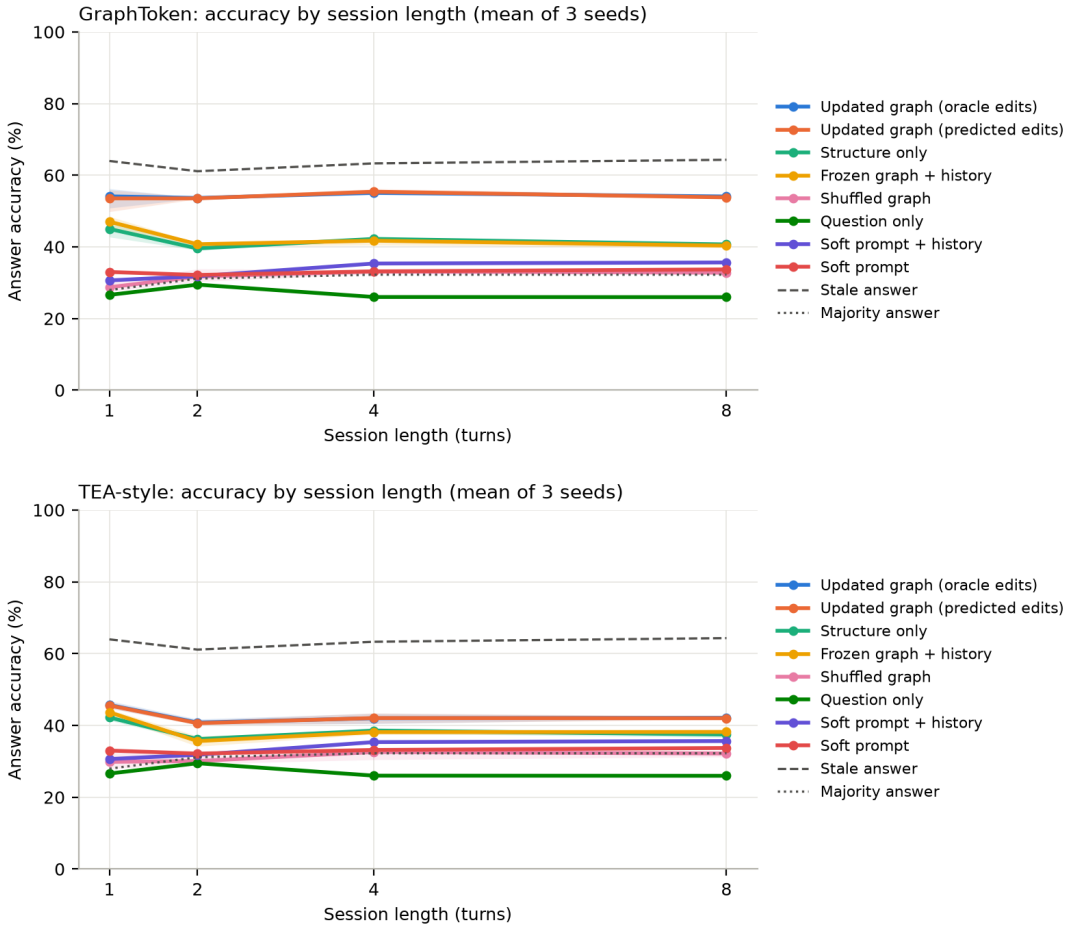


Figure 4: Test accuracy by session length for all conditions, GraphToken-style (top) and TEA-style (bottom).

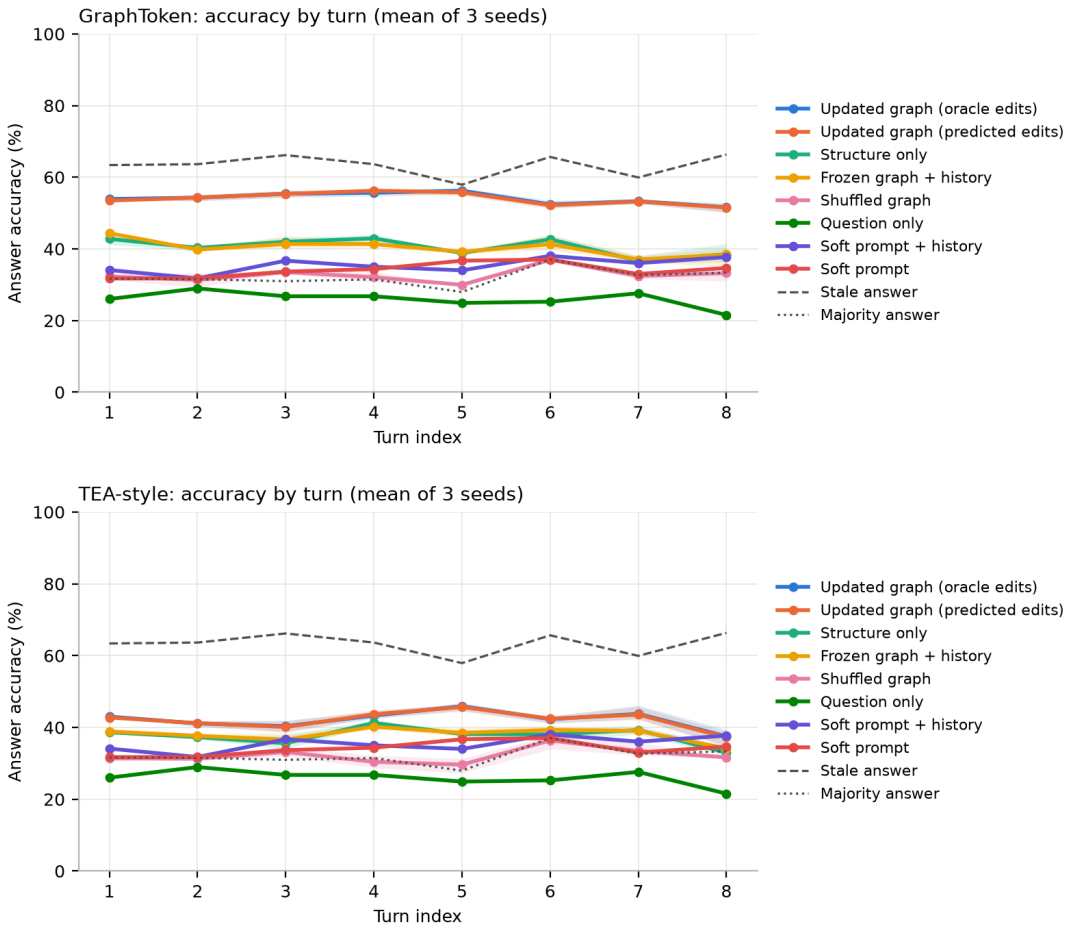


Figure 5: Test accuracy by turn index for all conditions, GraphToken-style (top) and TEA-style (bottom). Later turns occur only in 8-turn sessions.

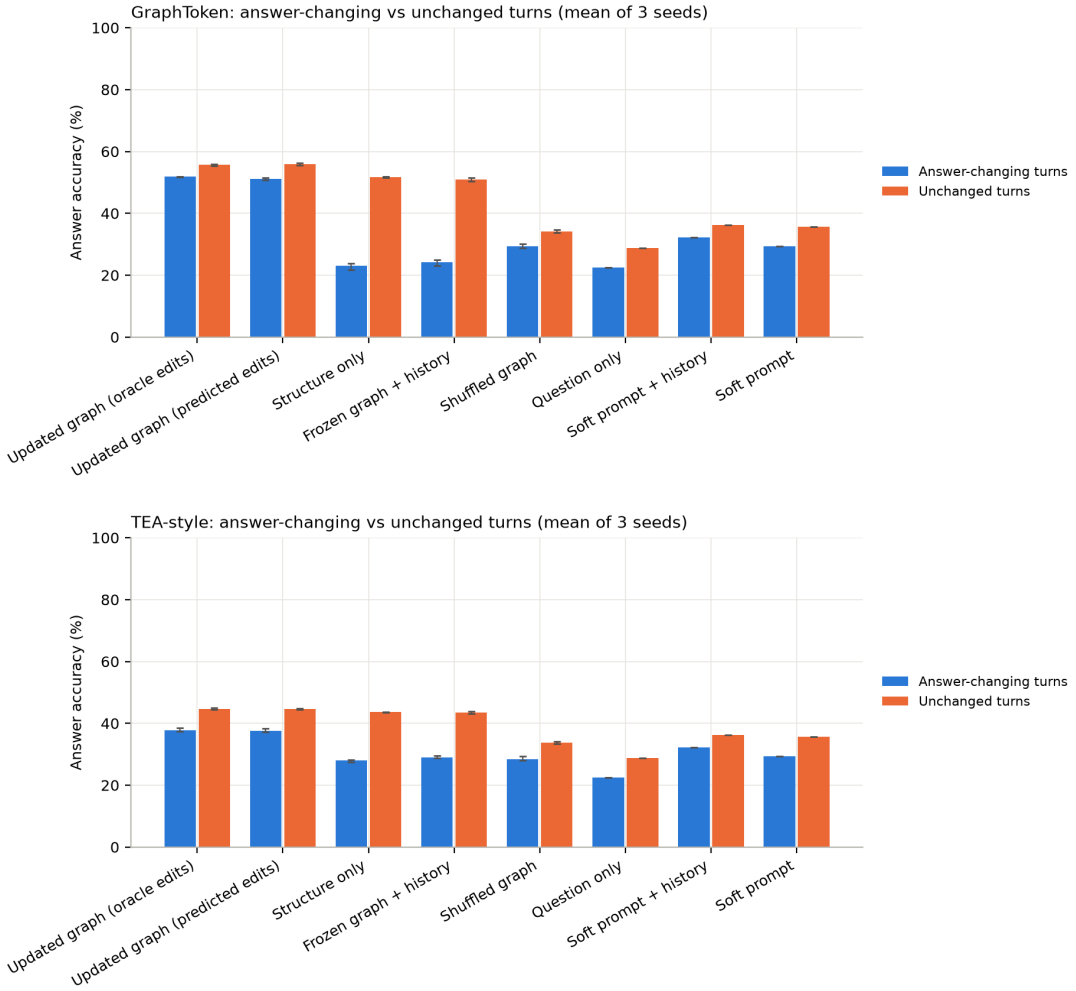


Figure 6: Test accuracy on answer-changing and unchanged turns for all conditions, GraphToken-style (top) and TEA-style (bottom).

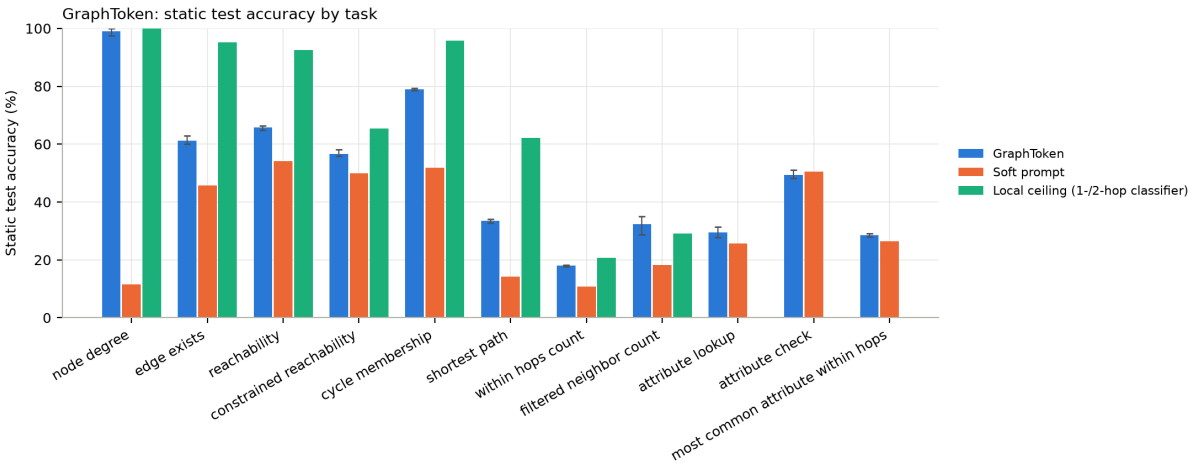


Figure 7: GraphToken-style static test accuracy on all eleven task families (mean of three seeds, error bars: seed range), with the soft prompt and the learned local ceiling (better of the one- and two-hop classifiers; structural tasks only).

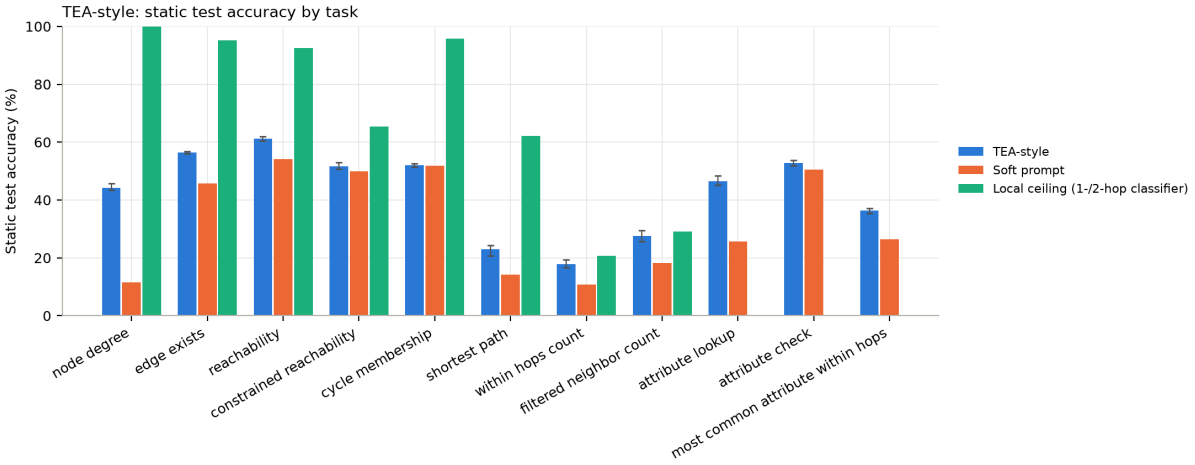


Figure 8: TEA-style static test accuracy on all eleven task families, laid out as in Figure 7.